SIMATS ENGINEERING
ASSESSMENT TOOL 2

COURSE NAME: Artificial Intelligence **COURSE CODE: CSA17**

COURSE OUTCOME COVERED

CO2: *Experiment search and game-solving techniques such as Greedy Search, A*, CSP, Minimax, and Alpha-Beta pruning with heuristic functions for solving complex AI problems efficiently. (BL3)*

Assessment Tool Weightage: 50%

Duration: 1 Hour

QUESTIONS: 5 **Total Marks:50**

Annexure A

Scenario Based Assignment

Code of Conduct:

I Chakradhar Peela 192412400(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.
Signature of the student:
Chakradhar Peela 192412400

Annexure A

Scenario Based Assignment

1. Scenario : A government deploys AI-powered drones to deliver emergency medicines in a flood-affected region. The environment is highly dynamic—roads may suddenly become inaccessible, and weather conditions affect travel cost. The drone has access to: A heuristic estimate (straight-line distance), Partial real-time updates about blocked paths and Variable movement costs depending on terrain and weather. However, due to urgency, the drone must quickly decide routes with minimum delay and risk.

Tasks:

i. Explain how Greedy Best-First Search would behave in this scenario and discuss its strengths and weaknesses under uncertainty

ii. Explain how A* would handle the same situation, including how it balances cost and heuristic

iii. Analyze the impact of heuristic accuracy on both algorithms

iv. Discuss how dynamic changes (blocked paths) affect search performance

v. Recommend the most suitable algorithm and justify your decision considering real-time constraints, optimality, and safety

2. **Scenario**: A smart city implements an AI system to optimize traffic signal timings across hundreds of intersections. The objective is to minimize Total waiting time, Fuel consumption and Traffic congestion. The system must continuously adjust signals based on traffic flow. However the search space is extremely large, traffic patterns change dynamically and the system may get stuck in locally optimal solutions

Tasks. Formulate this as an optimization problem and explain why traditional search is inefficient

i. Explain how Hill Climbing would work in this scenario and identify its limitations

ii. Discuss issues like local maxima, plateaus, and ridges with real-world interpretation

iii. Explain how Simulated Annealing or Genetic Algorithms can overcome these limitations

iv. Recommend the best approach and justify your choice based on scalability and adaptability

3. **Scenario:** An autonomous rover is deployed on Mars to explore unknown terrain. It must: navigate safely to collect samples, avoid hazards (craters, rocks), operate with limited sensing capability and make decisions without a complete map. The rover updates its knowledge as it explores, making this a real-time decision-making problem.

Tasks:

i. Explain why this problem requires an online search agent instead of offline search

ii. Analyze the challenges of partial observability and dynamic environments

iii. Describe how the rover builds and updates its internal model

iv. Compare online search with uninformed and informed search strategies

v. Justify the most suitable approach considering uncertainty, adaptability, and safety

4. **Scenario:** A university is designing an AI system to automatically generate exam timetables. The system must satisfy multiple constraints: No student should have overlapping exams, Limited number of exam halls, Certain subjects must be scheduled before others, Faculty availability constraints, Special accommodations for some students, The complexity increases as the number of courses and students grows.

Tasks :

Formulate the problem as a Constraint Satisfaction Problem (CSP)

i. Clearly define variables, domains, and constraints with explanation

ii. Explain how backtracking search works in this scenario

iii. Discuss how constraint propagation (e.g., forward checking) improves efficiency

iv. Analyze real-world challenges and justify the best strategy for scalability

5. **Scenario:** Strategic Game AI for Real-Time Decision Making (Minimax & Alpha-Beta Pruning) A gaming company is developing an AI opponent for a real-time strategy game. The AI must complete against human players, Make decisions within strict time limits, Evaluate complex game states with multiple possible moves, The decision tree is extremely large, making computation expensive.

Tasks:

i. Explain how the Minimax algorithm models this problem

ii. Analyze the computational challenges of large game trees

iii. Explain how Alpha-Beta pruning improves efficiency with detailed reasoning

iv. Design an evaluation function and justify the factors considered

v. Recommend a strategy for balancing optimality and real-time performance

RUBRICS FOR CALCULATION

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of Scenario	20%	Demonstrates clear and complete understanding of the scenario and context	Shows good understanding with minor gaps	Partial understanding; misses key aspects	Misinterprets or fails to understand the scenario
Identification of Key Issues	20%	Accurately identifies all relevant issues and factors involved	Identifies most key issues with minor omissions	Identifies limited issues; some irrelevant points	Unable to identify key issues
Application of Concepts	25%	Applies appropriate concepts effectively to address the scenario	Applies concepts with minor errors	Limited or partially correct application	Unable to apply relevant concepts
Decision Making	20%	Makes well-reasoned, logical decisions with strong rationale	Makes reasonable decisions with some justification	Makes weak decisions with limited justification	No clear decisions made or rationale provided
Clarity of Response	15%	Response is clear, structured, and logically presented	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand

1. **AI-Powered Drone Delivery in Flood-Affected Region (Greedy Best-First Search vs A*)**

i. Behavior of Greedy Best-First Search

Greedy Best-First Search selects the path that appears closest to the goal based only on the heuristic value:

$$f(n)=h(n)$$

where:

- $h(n)$= estimated distance from current node to destination.

Behavior in the drone scenario:

- The drone uses straight-line distance as the heuristic.
- It always chooses the route that seems geographically closest to the medical delivery point.
- It ignores actual travel cost, weather conditions, fuel consumption, and blocked roads.

Example:

A route may appear shortest from the air, but it may pass through:

- flooded areas,
- strong wind zones,
- blocked roads,
- dangerous terrain.

The drone may repeatedly choose such routes because they look closer.

Strengths:

- Very fast decision-making.
- Requires less computation.
- Useful when emergency response time is critical.
- Can quickly find a possible route.

Weaknesses:

- Does not guarantee the shortest or safest path.
- Easily misled by inaccurate heuristic estimates.
- Cannot handle changing costs effectively.
- May choose risky routes due to ignoring environmental conditions.

ii. A* Search in the Drone Scenario

A* combines actual cost and estimated cost:

$$f(n)=g(n)+h(n)$$

where:

- $g(n)$= cost already spent reaching the current location.
- $h(n)$= estimated remaining cost to destination.

Working:

The drone considers:

- distance travelled,
- weather impact,
- terrain difficulty,
- blocked paths,
- estimated remaining distance.

Example:

Two paths:

Path A

- Short distance
- Heavy storm
- High energy consumption

Path B

- Slightly longer
- Clear weather
- Lower risk

A* can select Path B because its total cost is lower.

Advantages:

- Produces optimal paths if heuristic is admissible.
- Balances speed and accuracy.
- Handles different movement costs better than Greedy Search.

Limitations:

- Requires more computation.
- Needs memory to store explored paths.
- Frequent environmental changes require repeated searching.

iii. Impact of Heuristic Accuracy

Greedy Best-First Search:

- Highly dependent on heuristic accuracy.
- A good heuristic leads to fast decisions.
- A poor heuristic causes wrong choices.

Example:
Straight-line distance ignores flood conditions, causing unsafe decisions.
A*:

- Also depends on heuristic quality.
- Accurate heuristic improves speed.
- Poor heuristic increases computation.
- If heuristic overestimates cost, optimality may be lost.

iv. Effect of Dynamic Changes
Blocked roads and weather changes create a changing search environment.
Effects:

- Previously calculated routes become invalid.
- The algorithm must recalculate paths.
- Search time increases.

Solutions:

- Dynamic A* algorithms.
- Real-time replanning.
- Incremental search methods.

v. Recommended Algorithm
*A with dynamic replanning is the most suitable.**
Reasons:

- Provides safer routes.
- Considers travel cost and risk.
- Can adapt to changing environments.
- Maintains near-optimal solutions.

For emergency delivery, a hybrid approach can be used:

- A* for planning.
- Real-time updates for correction.

2. Smart City Traffic Signal Optimization
Problem Formulation as Optimization
Objective:
Minimize:

$$Total\ \ Cost = Waiting\ \ Time + Fuel\ \ Consumption + Congestion$$

Variables:

- Signal timing duration.
- Green/red cycle lengths.
- Traffic priority rules.

Constraints:

- Maximum waiting time.
- Pedestrian crossing requirements.
- Emergency vehicle priority.

Why Traditional Search is Inefficient

The search space is extremely large because:

- Hundreds of intersections exist.
- Each signal has multiple possible timings.
- Traffic patterns continuously change.

Traditional exhaustive search:

- Requires checking every possible combination.
- Becomes computationally impossible.

i. Hill Climbing Approach

Hill Climbing starts with a possible traffic schedule and improves it step-by-step.

Process:

1. Generate initial signal timings.
2. Modify one intersection timing.
3. Evaluate improvement.
4. Keep changes that reduce congestion.

Example:

Increase green time on a busy road if it reduces waiting.

Limitations

- Only considers nearby solutions.
- Can stop before reaching the best solution.

ii. Local Maxima, Plateaus, and Ridges

Local Maximum

A solution better than nearby solutions but not globally optimal.

Example:

One intersection works perfectly, but overall city traffic remains poor.

Plateau
Many solutions have similar values.
Example:
Changing signal timing slightly gives no improvement.
Ridge
Best solutions require a combination of changes.
Example:
Improving one road requires adjusting several connected intersections.

iii. Simulated Annealing and Genetic Algorithms
Simulated Annealing
Inspired by metal cooling.
Features:

- Sometimes accepts worse solutions.
- Helps escape local maxima.
- Gradually focuses on better solutions.

Advantages:

- Good for complex optimization.
- Handles dynamic traffic.

Genetic Algorithm
Uses evolution concepts.
Steps:

1. Generate multiple traffic plans.
2. Evaluate fitness.
3. Select best solutions.
4. Combine and mutate them.
5. Repeat.

Advantages:

- Searches many areas simultaneously.
- Suitable for large-scale systems.

iv. Recommended Approach
Genetic Algorithm combined with real-time learning is most suitable.
Reasons:

- Handles thousands of intersections.
- Adapts to changing traffic.

- Avoids local optimum problems.
- Scales better than Hill Climbing.

3. Mars Autonomous Rover

i. Need for Online Search Agent

Offline search requires:

- Complete map.
- Known environment.
- Fixed conditions.

Mars exploration has:

- Unknown terrain.
- Limited sensors.
- Unexpected obstacles.

Therefore, the rover requires online search.

ii. Partial Observability and Dynamic Challenges

Problems:

Limited sensing:

The rover cannot see the entire environment.

Unknown hazards:

- Hidden rocks.
- Craters.
- Unsafe areas.

Communication delay:

Commands from Earth cannot be instant.

The rover must make independent decisions.

iii. Internal Model Updating

The rover maintains an internal representation:

- Known terrain map.
- Hazard locations.
- Safe paths.
- Sample locations.

Process:

1. Sense environment.
2. Update map.
3. Plan movement.

4. Execute action.
5. Repeat.

iv. Comparison of Search Strategies

Method	Characteristics
Uninformed Search	No environmental knowledge; inefficient
Informed Search	Uses heuristic; faster but requires information
Online Search	Learns while exploring

v. Suitable Approach

The best approach is:
Online informed search with adaptive planning.
Reasons:

- Handles unknown environments.
- Learns continuously.
- Improves safety.
- Supports real-time decisions.

4. University Exam Timetable as CSP

CSP Formulation
A Constraint Satisfaction Problem contains:

$$CSP = (V, a, riablesDomainsConstraints)$$

i. Variables

Each variable represents an exam.
Example:

$$X_1 = MathematicsExam\, X_2 = PhysicsExam$$

Domains
Possible time slots and halls.
Example:
Mathematics:
{Monday 9AM, Monday 2PM, Tuesday 9AM}

Constraints
Student constraint:
No overlapping exams.

Hall constraint:
Only available rooms can be assigned.
Faculty constraint:
Teachers must be available.
Ordering constraint:
Some exams must occur before others.
Accommodation constraint:
Special students require suitable timing.

ii. Backtracking Search
Process:

1. Assign a time slot to an exam.
2. Check constraints.
3. If valid, continue.
4. If conflict occurs, undo assignment.
5. Try another option.

Example:
Physics assigned Tuesday 9AM.
Conflict with Chemistry.
Backtrack and select another slot.

iii. Constraint Propagation
Forward checking:
After assigning one exam:

- Remove impossible choices from remaining exams.

Example:
If Hall A is occupied:

- Remove Hall A from other exams.

Benefits:

- Reduces search space.
- Detects conflicts early.

iv. Scalability Challenges
Challenges:

- Thousands of students.
- Many courses.
- Complex constraints.

Best strategy:
Backtracking + constraint propagation + heuristic ordering
Techniques:

- Minimum Remaining Values (MRV).
- Least Constraining Value.
- Arc consistency.

This provides better scalability.

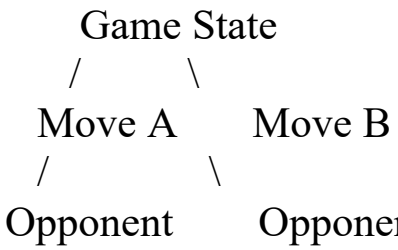
5. Strategic Game AI Using Minimax and Alpha-Beta Pruning

i. Minimax Model

Minimax assumes:

- AI tries to maximize winning chances.
- Opponent tries to minimize AI success.

Tree:



AI selects the move with maximum guaranteed outcome.

ii. Computational Challenges

Large games have:

- Huge branching factor.
- Millions of possible states.
- Limited decision time.

Complexity:

$$O(b^d)$$

where:

- b = number of possible moves.
- d = search depth.

iii. Alpha-Beta Pruning

Alpha-Beta removes branches that cannot influence the final decision.
Variables:
Alpha:
Best value found for maximizing player.
Beta:

Best value found for minimizing player.
If:

$$Alpha \geq Beta$$

remaining branches are ignored.
Benefits:

- Produces the same result as Minimax.
- Searches deeper.
- Reduces computation.

iv. Evaluation Function Design

Since complete search is impossible, an evaluation function estimates game states.
Example:

$$Evaluation = w_1(Material) + w_2(Position) + w_3(Resource) + w_4(Threats)$$

Factors:
Player advantage:

- Units.
- Territory.
- Resources.

Defensive factors:

- Safety.
- Remaining health.

Strategic factors:

- Future opportunities.
- Enemy weakness.

v. Recommended Strategy

Use:
Depth-limited Minimax + Alpha-Beta pruning + heuristic evaluation
Reasons:

- Meets real-time requirements.
- Provides strong decisions.
- Reduces computation.
- Balances optimality and speed.

Additional improvements:

- Move ordering.
- Transposition tables.
- Machine learning evaluation models.